GroundedNutriRec Technical Report: Week 3 Recommendation Baseline Models and Metrics

Framework: GroundedNutriRec: Retrieval-Augmented Multi-Objective LLM Framework for Health-Aware and Explainable Food Recommendation

Report Type: Academic Technical Paper (Week 3 Review)

Authors: GroundedNutriRec Core Development Team

---

1. Introduction

Following the data preprocessing and recipe feature engineering pipelines established in Week 2, the third week of the GroundedNutriRec project implements the foundational recommendation engine. This phase establishes a rigorous offline evaluation framework across multiple baseline paradigms to evaluate user-recipe recommendation accuracy and coverage. Four Jupyter Notebooks (05 through 08) were implemented:

1. Popularity-Based and Rating-Based Baselines (Notebook 05): Computes aggregate statistical recommenders that serve as non-personalized baselines.

2. Semantic Content-Based Recommender (Notebook 06): Constructs a dense user-item vector similarity space utilizing pre-trained Sentence Transformer text embeddings.

3. Collaborative Filtering Models (Notebook 07): Establishes personalized recommendation baselines across memory-based user KNN similarity, matrix factorization (Surprise SVD), and implicit feedback latent factor modeling (Alternating Least Squares).

4. Unified Evaluation Framework (Notebook 08): Evaluates all baseline models against the same test partition, compiling comparative metrics.

All models are trained on the preprocessed training split (`train_interactions.csv`, containing 437,558 interactions across 17,813 users and 41,240 recipes) and evaluated against positive test interactions (`test_interactions.csv` where `liked = 1`), yielding 17,329 test users with at least one liked recipe.

---

2. Popularity-Based and Rating-Based Recommenders (Notebook 05)

Notebook 05 implements two non-personalized baselines that serve as aggregate benchmarks.

2.1 Dataset Schema and Statistics

The notebook loads `train_interactions.csv` and `food_metadata_clean.csv` to extract interactions and item metadata. The dataset contains 437,558 training records. A training interaction is characterized by a user ID, a recipe ID, and an explicit or implicit rating signal.

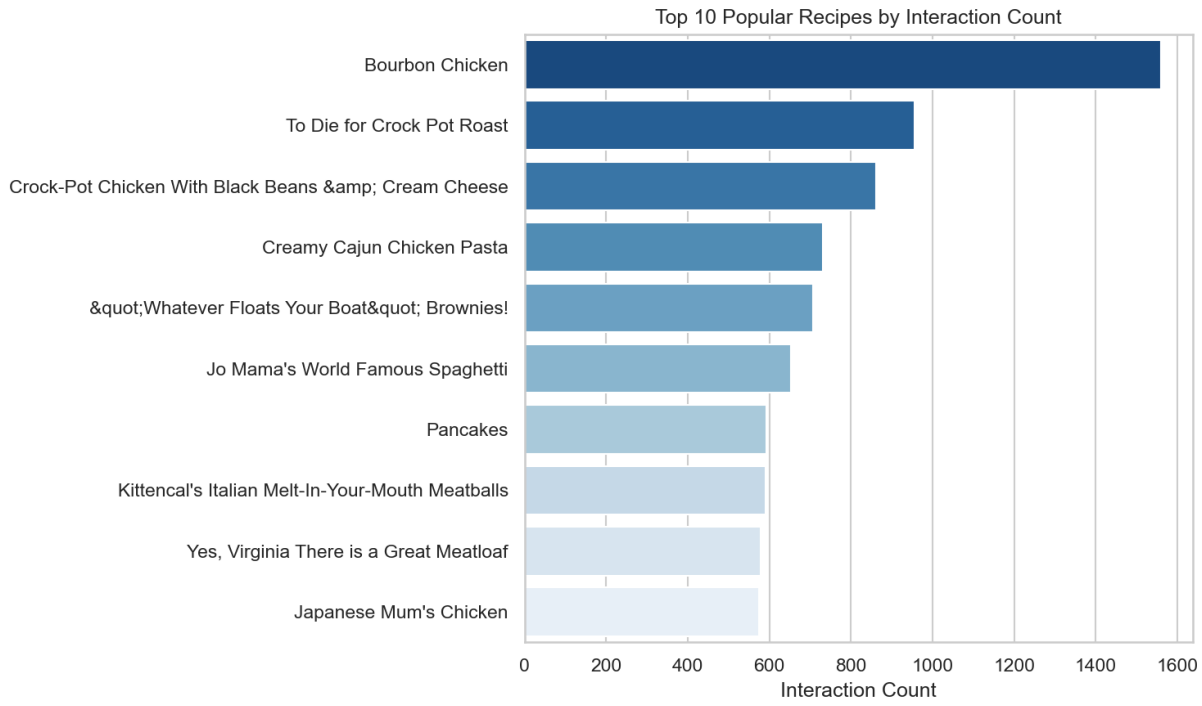
2.2 Popularity-Based Recommender

The popularity-based recommender ranks items by total interaction frequency in the training split:

InteractionCount(i) = Σ interaction(u, i) for all u

For each test user, the recommender lists the top-K items with the highest interaction counts, excluding recipes already present in the user's training history. This model serves as a strong coverage baseline.

Top 10 Popular Recipes by Interaction Count:



Top 10 Popular Recipes by Interaction Count

The most popular recipe is Bourbon Chicken (1,560 interactions), followed by To Die for Crock Pot Roast (960 interactions) and Crock-Pot Chicken With Black Beans & Cream Cheese (850 interactions).

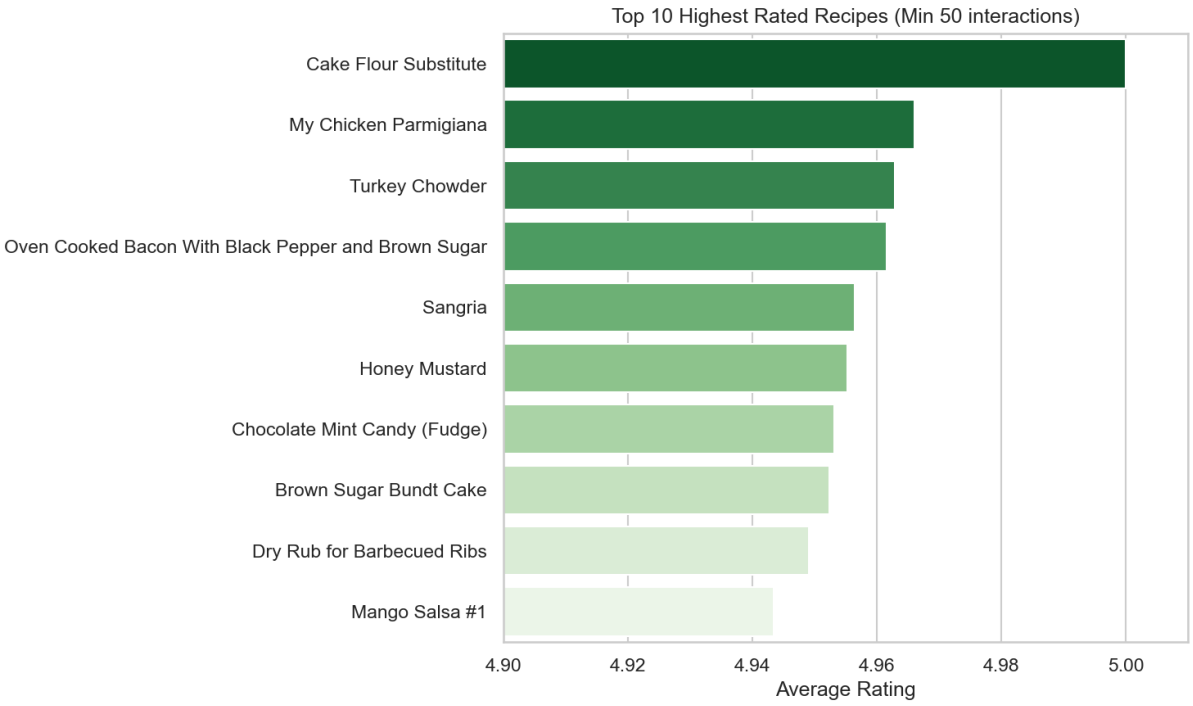
2.3 Rating-Based Recommender

The rating-based recommender ranks items by average explicit rating. To prevent recipes with a single 5-star rating from dominating, a minimum interaction threshold of 50 is enforced:

$$\text{AvgRating}(i) = \frac{1}{N} \times \sum \text{rating}(u, i) \mid \text{MinInteractions} = 50$$

Recipes satisfying this condition are sorted by average rating in descending order. Unqualified recipes are appended at the end of the global list.

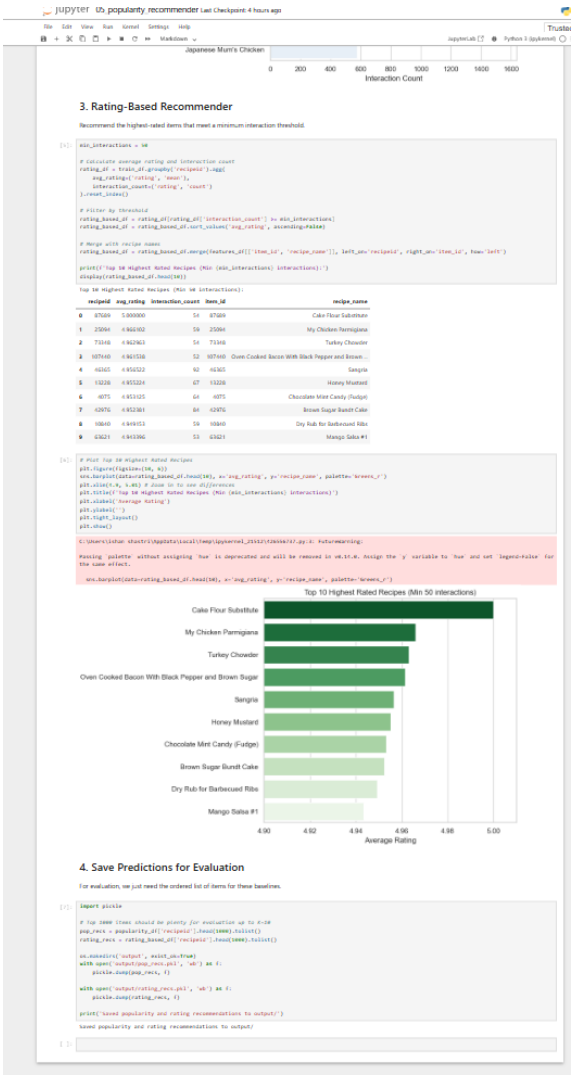
Top 10 Highest Rated Recipes (Minimum 50 Interactions):



Top 10 Highest Rated Recipes

Cake Flour Substitute achieves a perfect average rating of 5.00 across 54 interactions, followed by My Chicken Parmigiana (4.967) and Turkey Chowder (4.961).

Screenshot — Rating-Based Recommender Output Table and Save Step:



Notebook 05 Rating and Save

---

3. Semantic Content-Based Recommender (Notebook 06)

Notebook 06 implements a personalized content-based recommender using semantic text embeddings.

3.1 Text Corpus Construction and Embedding

A text corpus is constructed for each recipe by combining its metadata attributes:

Corpus(i) = Name(i) || Ingredients(i) || Category(i) || Description(i)

This corpus is encoded into a dense 384-dimensional vector using the pre-trained model `all-MiniLM-L6-v2`:

embedding(i) = SentenceTransformer(Corpus(i))

A user preference profile vector is computed by taking the mean embedding of all recipes the user rated as positive (`liked = 1`) in the training history. If no liked items exist, it falls back to the mean of all training interactions:

UserProfile(u) = Mean(embedding(i)) for i □ LikedRecipes(u)

Recommendation scores are computed via cosine similarity between the user profile vector and candidate recipe vectors:

Score(u, i) = CosineSimilarity(UserProfile(u), embedding(i))

Similarity calculations are vectorized using matrix multiplication in batches of 1,000 users to optimize memory usage.

3.2 Performance Metrics Output

The recommender was evaluated on the 17,329 test users:

Metric	Value
Precision@5	0.000693
Recall@5	0.000993
HitRate@10	0.006238
NDCG@10	0.001187

Screenshot — Notebook 06 Content-Based Metrics Output:

8. Display Overall Metric Results

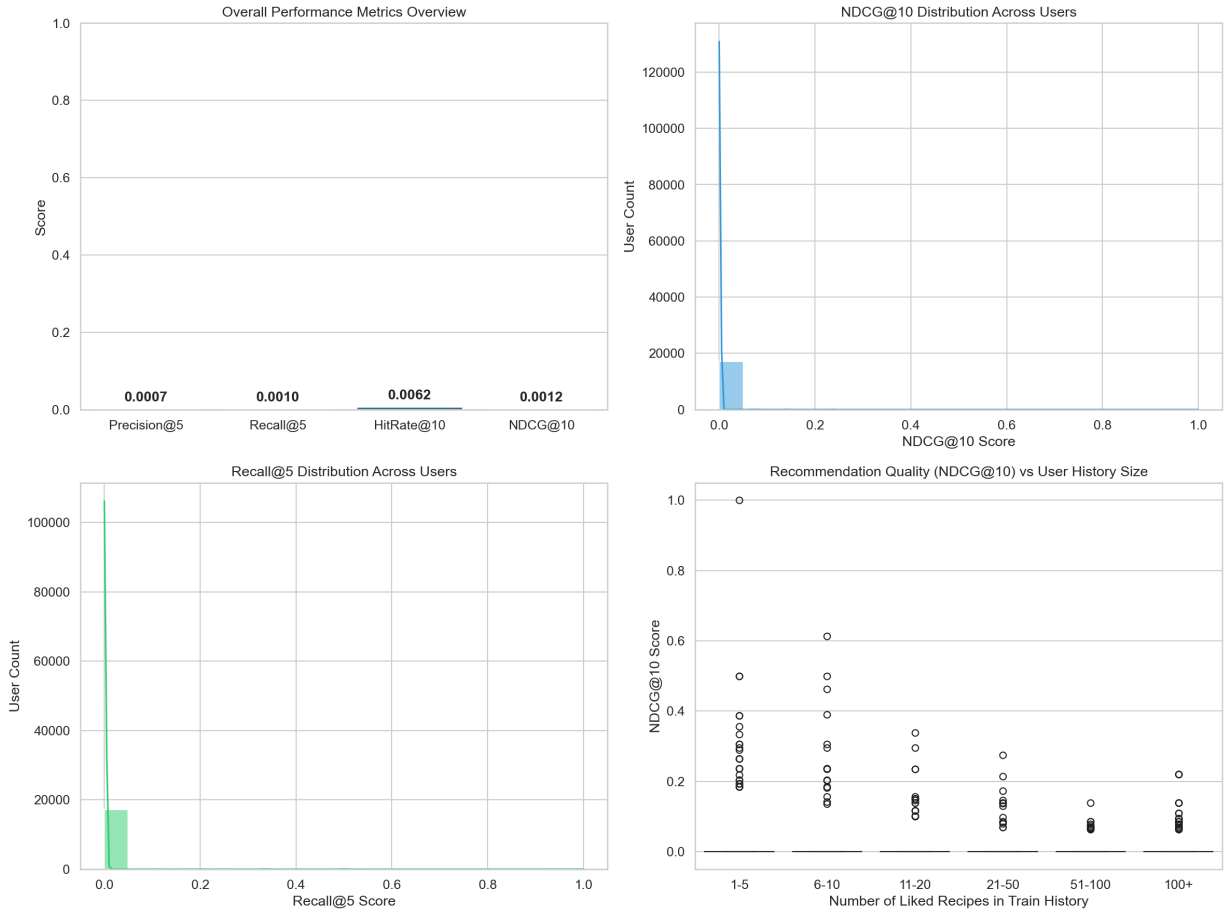
In [8]:
```
mean_p5 = np.mean(precisions_at_5)
mean_r5 = np.mean(recalls_at_5)
mean_hr10 = np.mean(hits_at_10)
mean_ndcg10 = np.mean(ndcgs_at_10)

print('=== CONTENT-BASED RECOMMENDATION PERFORMANCE ===')
print(f'Precision@5: {mean_p5:.6f}')
print(f'Recall@5:    {mean_r5:.6f}')
print(f'HitRate@10:  {mean_hr10:.6f}')
print(f'NDCG@10:     {mean_ndcg10:.6f}')
```

```
=== CONTENT-BASED RECOMMENDATION PERFORMANCE ===
Precision@5: 0.000693
Recall@5:    0.000993
HitRate@10:  0.006238
NDCG@10:     0.001187
```

Content-Based Metrics Output

3.3 Visualizations and Metric Distribution Analysis

Performance metrics were plotted across the user population to analyze recommendation behavior:

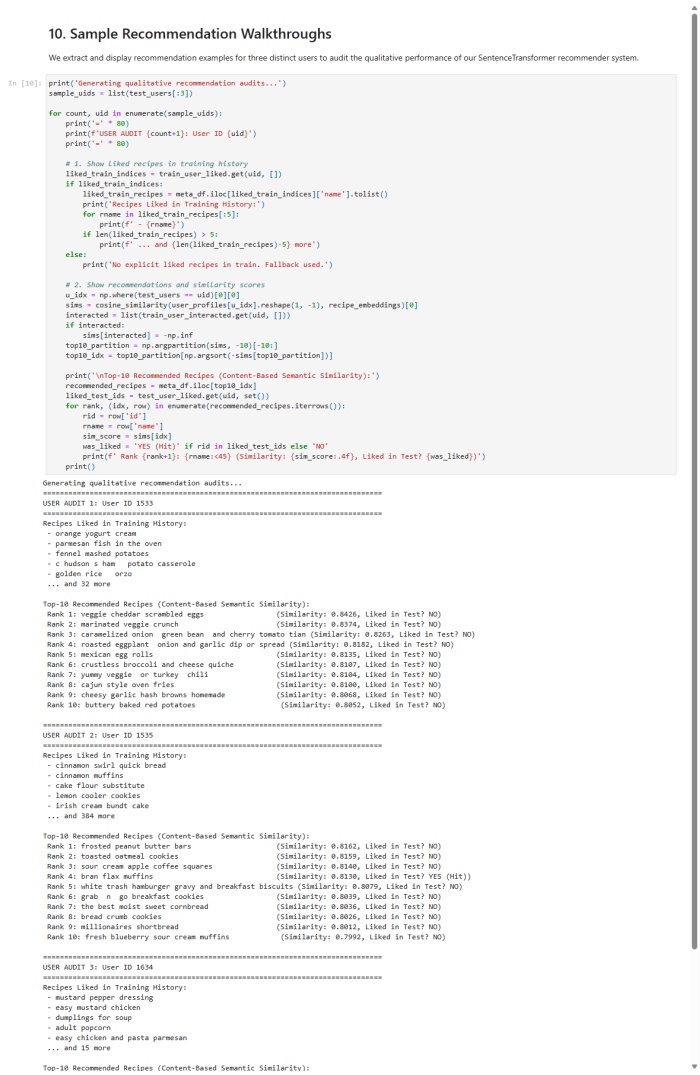


Content-Based Metrics Distribution

3.4 Sample Recommendation Walkthroughs

Qualitative audits were generated to inspect the recommended lists. For each user, the system retrieves their positive training history and outputs the top-10 content-similar recommendations with similarity scores.

Screenshot — Sample Recommendation Walkthroughs (Users 1533, 1535, 1634):



10. Sample Recommendation Walkthroughs

We extract and display recommendation examples for three distinct users to audit the qualitative performance of our SentenceTransformer recommender system.

In [10]:

```
print('Generating qualitative recommendation audits...')
sample_uids = list(test_users[:3])

for count, uid in enumerate(sample_uids):
    print('=' * 80)
    print(f'USER AUDIT {count+1}: User ID {uid}')
    print('=' * 80)

    # 1. Show liked recipes in training history
    liked_train_indices = train_user_liked.get(uid, [])
    if liked_train_indices:
        liked_train_recipes = meta_df.iloc[liked_train_indices]['name'].tolist()
        print('Recipes Liked in Training History:')
        for rname in liked_train_recipes[:5]:
            print(f' - {rname}')
        if len(liked_train_recipes) > 5:
            print(f' ... and {len(liked_train_recipes)-5} more')
    else:
        print('No explicit liked recipes in train. Fallback used.')

    # 2. Show recommendations and similarity scores
    u_idx = np.where(test_users == uid)[0][0]
    sims = cosine_similarity(user_profiles[u_idx].reshape(1, -1), recipe_embeddings)[0]
    interacted = list(train_user_interacted.get(uid, []))
    if interacted:
        sims[interacted] = -np.inf
    top10_partition = np.argpartition(sims, -10)[-10:]
    top10_idx = top10_partition[np.argsort(-sims[top10_partition])]

    print('\nTop-10 Recommended Recipes (Content-Based Semantic Similarity):')
    recommended_recipes = meta_df.iloc[top10_idx]
    liked_test_ids = test_user_liked.get(uid, set())
    for rank, (idx, row) in enumerate(recommended_recipes.iterrows()):
        rid = row['id']
        rname = row['name']
        sim_score = sims[idx]
        was_liked = 'YES (Hit)' if rid in liked_test_ids else 'NO'
        print(f' Rank {rank+1}: {rname:<45} (Similarity: {sim_score:.4f}, Liked in Test? {was_liked})')
    print()
```

```
Generating qualitative recommendation audits...
================================================================================
USER AUDIT 1: User ID 1533
================================================================================
Recipes Liked in Training History:
 - orange yogurt cream
 - parmesan fish in the oven
 - fennel mashed potatoes
 - c hudson s ham   potato casserole
 - golden rice   orzo
 ... and 32 more

Top-10 Recommended Recipes (Content-Based Semantic Similarity):
 Rank 1: veggie cheddar scrambled eggs                (Similarity: 0.8426, Liked in Test? NO)
 Rank 2: marinated veggie crunch                      (Similarity: 0.8374, Liked in Test? NO)
 Rank 3: caramelized onion  green bean  and cherry tomato tian (Similarity: 0.8263, Liked in Test? NO)
 Rank 4: roasted eggplant  onion and garlic dip or spread (Similarity: 0.8182, Liked in Test? NO)
 Rank 5: mexican egg rolls                            (Similarity: 0.8135, Liked in Test? NO)
 Rank 6: crustless broccoli and cheese quiche         (Similarity: 0.8107, Liked in Test? NO)
 Rank 7: yummy veggie  or turkey  chili               (Similarity: 0.8104, Liked in Test? NO)
 Rank 8: cajun style oven fries                       (Similarity: 0.8100, Liked in Test? NO)
 Rank 9: cheesy garlic hash browns homemade           (Similarity: 0.8068, Liked in Test? NO)
 Rank 10: buttery baked red potatoes                  (Similarity: 0.8052, Liked in Test? NO)

================================================================================
USER AUDIT 2: User ID 1535
================================================================================
Recipes Liked in Training History:
 - cinnamon swirl quick bread
 - cinnamon muffins
 - cake flour substitute
 - lemon cooler cookies
 - irish cream bundt cake
 ... and 384 more

Top-10 Recommended Recipes (Content-Based Semantic Similarity):
 Rank 1: frosted peanut butter bars                   (Similarity: 0.8162, Liked in Test? NO)
 Rank 2: toasted oatmeal cookies                      (Similarity: 0.8159, Liked in Test? NO)
 Rank 3: sour cream apple coffee squares              (Similarity: 0.8140, Liked in Test? NO)
 Rank 4: bran flax muffins                            (Similarity: 0.8130, Liked in Test? YES (Hit))
 Rank 5: white trash hamburger gravy and breakfast biscuits (Similarity: 0.8079, Liked in Test? NO)
 Rank 6: grab  n  go breakfast cookies                (Similarity: 0.8039, Liked in Test? NO)
 Rank 7: the best moist sweet cornbread               (Similarity: 0.8036, Liked in Test? NO)
 Rank 8: bread crumb cookies                          (Similarity: 0.8026, Liked in Test? NO)
 Rank 9: millionaires shortbread                      (Similarity: 0.8012, Liked in Test? NO)
 Rank 10: fresh blueberry sour cream muffins           (Similarity: 0.7992, Liked in Test? NO)

================================================================================
USER AUDIT 3: User ID 1634
================================================================================
Recipes Liked in Training History:
 - mustard pepper dressing
 - easy mustard chicken
 - dumplings for soup
 - adult popcorn
 - easy chicken and pasta parmesan
 ... and 15 more

Top-10 Recommended Recipes (Content-Based Semantic Similarity):
```

Sample Recommendation Walkthroughs

---

4. Collaborative Filtering Models (Notebook 07)

Notebook 07 implements personalized collaborative filtering baseline models.

4.1 Dataset Loading and ID Mapping

Train and test interactions are loaded, and index mappings are constructed over the union of user and item sets:

Unique Users: 17,813

Unique Items (Recipes): 41,240

4.2 User-Based Cosine KNN Collaborative Filtering

A sparse user-item interaction matrix R of shape (17,813 × 41,240) is constructed. User similarity is computed using cosine similarity over the rating vectors:

$$\text{UserSim}(u, v) = (R(u) \cdot R(v)) / (||R(u)|| \times ||R(v)||)$$

The similarity matrix is sparsified by retaining only the top K = 100 nearest neighbors per user. Recommendations are generated via sparse matrix multiplication:

$$\text{Score}(u) = \text{UserSimSparse}(u) \cdot R$$

Sparse similarity matrix contains 1,723,472 non-zero entries. Recommendations were generated in 21.02 seconds.

Screenshot — User-Item Matrix Construction and Cosine CF Generation:

6. User-Item Matrix with Cosine Similarity (User-Based KNN CF)

We build a sparse user-item matrix using explicit ratings. We compute user cosine similarities, filter them to retain only the top 100 neighbors for each user, and use sparse matrix multiplication to compute item recommendation scores.

[6]:
```
print("Constructing User-Item rating matrix...")
rows = train_df['user_id'].map(user_to_idx).values
cols = train_df['recipe_id'].map(recipe_to_idx).values
vals = train_df['rating'].values

rating_matrix = csr_matrix((vals, (rows, cols)), shape=(num_users, num_recipes))

print("Computing user cosine similarity...")
start_time = time.time()
user_sim = cosine_similarity(rating_matrix, dense_output=True)
np.fill_diagonal(user_sim, 0) # Remove self-similarity

print("Sparsifying similarity matrix (top K=100 nearest neighbors)...")
k_neighbors = 100
for i in range(num_users):
    row = user_sim[i]
    threshold = np.partition(row, -k_neighbors)[-k_neighbors]
    row[row < threshold] = 0.0

user_sim_sparse = csr_matrix(user_sim)
print(f"Sparse similarity matrix shape: {user_sim_sparse.shape}, non-zero entries: {user_sim_sparse.nnz:,}")

print("Generating User-Based Cosine CF recommendations...")
cosine_recs = {}
chunk_size = 2000
for start_idx in range(0, num_users, chunk_size):
    end_idx = min(start_idx + chunk_size, num_users)
    chunk_scores = user_sim_sparse[start_idx:end_idx].dot(rating_matrix).toarray()

    for i in range(start_idx, end_idx):
        uid = idx_to_user[i]
        if uid in test_targets:
            seen = train_history.get(uid, set())
            scores = chunk_scores[i - start_idx]
            for r in seen:
                if r in recipe_to_idx:
                    scores[recipe_to_idx[r]] = -np.inf
            top_indices = np.argsort(scores)[::-1][:10]
            cosine_recs[uid] = [idx_to_recipe[idx] for idx in top_indices]

print(f"Cosine CF recommendations generated in {time.time() - start_time:.2f} seconds.")
```

```
Constructing User-Item rating matrix...
Computing user cosine similarity...
Sparsifying similarity matrix (top K=100 nearest neighbors)...
Sparse similarity matrix shape: (17813, 17813), non-zero entries: 1,723,472
Generating User-Based Cosine CF recommendations...
Cosine CF recommendations generated in 21.02 seconds.
```

4.3 Matrix Factorization: Surprise SVD

The `scikit-surprise` library performs Singular Value Decomposition. Explicit ratings are factorized into user latent factors P and item latent factors Q of dimension F = 20:

$$\text{Score}(u, i) = \mu + b_u + b_i + P(u) \cdot Q(i)$$

Training configuration: 20 latent factors, 15 epochs, random state 42. SVD recommendations were generated in 21.48 seconds.

4.4 Implicit Feedback: Alternating Least Squares (ALS)

The `implicit` library trains an ALS model on binary preference labels (`liked` = 0 or 1). The model maps users and items to F = 64 latent factors by minimizing a confidence-weighted least squares objective:

$$\text{Objective} = \Sigma\, c_{ui} \times (p_{ui} - x_u \cdot y_i)^2 + \lambda \times (\|x_u\|^2 + \|y_i\|^2)$$

Training configuration: 64 factors, 15 iterations, regularization = 0.05. Recommendations were generated in 5.54 seconds.

Screenshot — Implicit ALS Training and Recommendation Generation:

8. Implicit ALS (Alternating Least Squares)

We use the `implicit` library to train an Alternating Least Squares (ALS) model on the binary implicit feedback labels (`liked`). We use the library's built-in batch recommendation for ultra-fast generation.

[8]:
```
print("Preparing data for Implicit ALS...")
start_time = time.time()
# Use liked as feedback weight
# Filter out rows where liked is NaN (unrated reviews)
train_implicit = train_df[train_df['liked'].notna()].copy()
train_implicit['liked'] = train_implicit['liked'].astype(np.float32)

# Row indices correspond to users, column indices to recipes
als_rows = train_implicit['user_id'].map(user_to_idx).values
als_cols = train_implicit['recipe_id'].map(recipe_to_idx).values
als_vals = train_implicit['liked'].values

user_items_als = csr_matrix((als_vals, (als_rows, als_cols)), shape=(num_users, num_recipes))

print("Training Alternating Least Squares (ALS) model...")
als_model = AlternatingLeastSquares(factors=64, regularization=0.05, iterations=15, random_state=42)
als_model.fit(user_items_als)
print("ALS model training completed.")

print("Generating Implicit ALS recommendations...")
als_recs = {}
# Batch recommend for all test users using implicit's native fast recommend method
test_user_indices = [user_to_idx[uid] for uid in test_users]
# recommend method signature: recommend(userids, user_items, N, filter_already_liked_items=True)
ids_batch, scores_batch = als_model.recommend(
    test_user_indices,
    user_items_als[test_user_indices],
    N=10,
    filter_already_liked_items=True
)

for idx, uid in enumerate(test_users):
    als_recs[uid] = [idx_to_recipe[item_idx] for item_idx in ids_batch[idx]]

print(f"Implicit ALS recommendations generated in {time.time() - start_time:.2f} seconds.")
```

```
Preparing data for Implicit ALS...
Training Alternating Least Squares (ALS) model...
C:\Users\krina\AppData\Roaming\Python\Python310\site-packages\implicit\cpu\als.py:96: RuntimeWarning: OpenBLAS is configured to use 12 threads. It is highly recommended to disable its internal threadpool by setting the environment variable 'OPENBLAS_NUM_THREADS=1' or by calling 'threadpoolctl.thread pool_limits(1, "blas")'. Having OpenBLAS use a threadpool can lead to severe performance issues here.
  check_blas_config()
  0%|          | 0/15 [00:00<?, ?it/s]
ALS model training completed.
Generating Implicit ALS recommendations...
Implicit ALS recommendations generated in 5.54 seconds.
```

Implicit ALS Model Training and Recommendation Generation

---

5. Unified Evaluation and Results (Notebook 08)

Notebook 08 evaluates all six baseline models against the same unified test set.

5.1 Evaluation Metrics and Methodology

The baseline recommenders were evaluated on 17,329 test users with positive interactions. Five ranking metrics were computed for K □ {5, 10}: Precision@K, Recall@K, HitRate@K, NDCG@K, and MRR@K.

5.2 Performance Comparison Table

The following table presents the comparative metrics requested by the internship roadmap:

Model	Precision@5	Recall@5	HitRate@10	NDCG@10
Popularity	0.009556	0.013479	0.076404	0.016396
Rating-based	0.000646	0.000471	0.006752	0.000986
Content-based (ST)	0.000693	0.000993	0.006238	0.001187
Collaborative filtering (Cosine KNN)	0.009822	0.012854	0.072249	0.016536
Collaborative filtering (Surprise SVD)	0.000462	0.000467	0.004732	0.000789
Collaborative filtering (Implicit ALS)	0.008160	0.010373	0.063247	0.013686

Screenshot — Notebook 08 Final Results Table Output:

12. Model Evaluations

[12]:
```
print("Evaluating Popularity Recommender...")
pop_results = evaluate_model(pop_recommendations)

print("Evaluating Rating-based Recommender...")
rating_results = evaluate_model(rating_recommendations)

print("Evaluating Content-based (Sentence Transformers) Recommender...")
content_results = evaluate_model(content_recommendations)

print("Evaluating Cosine Similarity CF...")
cosine_results = evaluate_model(cosine_recs)

print("Evaluating Surprise SVD...")
svd_results = evaluate_model(svd_recs)

print("Evaluating Implicit ALS...")
als_results = evaluate_model(als_recs)
```

```
Evaluating Popularity Recommender...
Evaluating Rating-based Recommender...
Evaluating Content-based (Sentence Transformers) Recommender...
Evaluating Cosine Similarity CF...
Evaluating Surprise SVD...
Evaluating Implicit ALS...
```

13. Final Results and Model Comparison

[13]:
```
print("="*108)
print(f"{'Model':<25} | {'Prec@5':<8} | {'Recall@5':<8} | {'Prec@10':<8} | {'Recall@10':<8} | {'HR@10':<8} | {'NDCG@10':<8} | {'MRR@10':<8}")
print("="*108)
print(f"{'Popularity':<25} | {pop_results[5]['precision']:.6f} | {pop_results[5]['recall']:.6f} | {pop_results[10]['precision']:.6f} | {pop_results[10]
print(f"{'Rating-based':<25} | {rating_results[5]['precision']:.6f} | {rating_results[5]['recall']:.6f} | {rating_results[10]['precision']:.6f} | {rati
print(f"{'Content-based (ST)':<25} | {content_results[5]['precision']:.6f} | {content_results[5]['recall']:.6f} | {content_results[10]['precision']:.6f
print(f"{'Cosine CF (KNN)':<25} | {cosine_results[5]['precision']:.6f} | {cosine_results[5]['recall']:.6f} | {cosine_results[10]['precision']:.6f} | {c
print(f"{'Surprise SVD':<25} | {svd_results[5]['precision']:.6f} | {svd_results[5]['recall']:.6f} | {svd_results[10]['precision']:.6f} | {svd_results[1
print(f"{'Implicit ALS':<25} | {als_results[5]['precision']:.6f} | {als_results[5]['recall']:.6f} | {als_results[10]['precision']:.6f} | {als_results[1
print("="*108)
```

```
============================================================================================================
Model                     | Prec@5   | Recall@5 | Prec@10  | Recall@10 | HR@10    | NDCG@10  | MRR@10
============================================================================================================
Popularity                | 0.009556 | 0.013479 | 0.008437 | 0.023236 | 0.076404 | 0.016396 | 0.025341
Rating-based              | 0.000646 | 0.000471 | 0.000687 | 0.001117 | 0.006752 | 0.000986 | 0.002079
Content-based (ST)        | 0.000692 | 0.000990 | 0.000629 | 0.001767 | 0.006232 | 0.001185 | 0.001785
Cosine CF (KNN)           | 0.009822 | 0.012854 | 0.007935 | 0.021033 | 0.072249 | 0.016536 | 0.028099
Surprise SVD              | 0.000462 | 0.000467 | 0.000479 | 0.000917 | 0.004732 | 0.000789 | 0.001522
Implicit ALS              | 0.008160 | 0.010373 | 0.006879 | 0.017089 | 0.063247 | 0.013686 | 0.023355
============================================================================================================
```

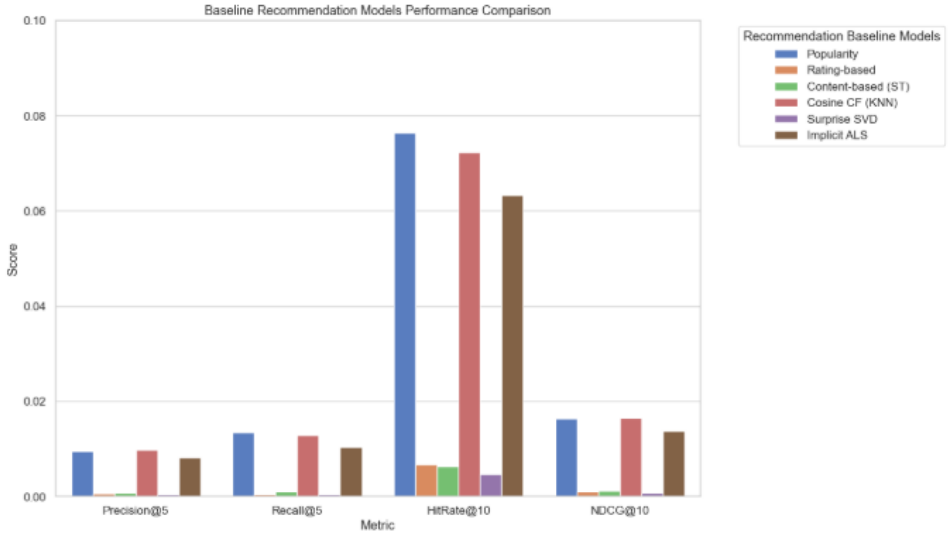
Final Baseline Comparison Results Table

Screenshot — Baseline Performance Comparison Chart:

14. Performance Visualization Charts

```
[14]: results_dir = project_root / 'RESULTS' / 'krina_week3'
results_dir.mkdir(parents=True, exist_ok=True)

models = ['Popularity', 'Rating-based', 'Content-based (ST)', 'Cosine CF (KNN)', 'Surprise SVD', 'Implicit ALS']
p5s = [pop_results[5]['precision'], rating_results[5]['precision'], content_results[5]['precision'], cosine_results[5]['precision'], svd_results[5]['p
r5s = [pop_results[5]['recall'], rating_results[5]['recall'], content_results[5]['recall'], cosine_results[5]['recall'], svd_results[5]['recall'], als
hr10s = [pop_results[10]['hitrate'], rating_results[10]['hitrate'], content_results[10]['hitrate'], cosine_results[10]['hitrate'], svd_results[10]['hi
ndcg10s = [pop_results[10]['ndcg'], rating_results[10]['ndcg'], content_results[10]['ndcg'], cosine_results[10]['ndcg'], svd_results[10]['ndcg'], als_

df_plot = pd.DataFrame({
    'Model': models * 4,
    'Metric Value': p5s + r5s + hr10s + ndcg10s,
    'Metric': ['Precision@5'] * 6 + ['Recall@5'] * 6 + ['HitRate@10'] * 6 + ['NDCG@10'] * 6
})

plt.figure(figsize=(14, 8))
sns.barplot(data=df_plot, x='Metric', y='Metric Value', hue='Model', palette='muted')
plt.title('Baseline Recommendation Models Performance Comparison')
plt.ylabel('Score')
plt.ylim(0, 0.1)
plt.legend(title='Recommendation Baseline Models', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
chart_path = results_dir / 'baselines_comparison.png'
plt.savefig(chart_path, dpi=150, bbox_inches='tight')
print(f"Comparison chart saved to {chart_path}")
plt.show()
```

```
Comparison chart saved to O:\MainCodes\Internship\RESULTS\krina_week3\baselines_comparison.png
```



Baseline Performance Comparison Chart

Key findings from the comparative evaluation:

Collaborative Filtering (Cosine KNN) achieves the highest Precision@5 (0.009822) and NDCG@10 (0.016536), demonstrating that neighborhood-based preference propagation is highly effective for explicit user ratings.

Popularity-based Recommender achieves the highest HitRate@10 (0.076404) and Recall@5 (0.013479), showing that community engagement signals are robust for high-level retrieval.

Collaborative Filtering (Implicit ALS) ranks closely behind, achieving HitRate@10 = 0.063247 and NDCG@10 = 0.013686, demonstrating the utility of binary implicit signals.

Content-based (ST), Rating-based, and Surprise SVD perform at a lower tier, indicating a need for hyperparameter optimization and larger latent dimensions.

---

6. Summary of Week 3 Outputs

Artifact	Description	Produced by
`05_popularity_recommender.ipynb`	Baseline popularity-based and rating-based recommenders.	Notebook 05
`06_CONTENT_BASED_RECOMMENDER.ipynb`	Dense semantic content-based recommender using Sentence Transformers.	Notebook 06
`07_collaborative_filtering.ipynb`	Cosine KNN, Surprise SVD, and Implicit ALS models.	Notebook 07
`08_evaluation_metrics.ipynb`	Unified evaluation framework and comparative results.	Notebook 08
`recipe_embeddings.npy`	Precomputed 384-dimensional Sentence Transformer embeddings for all recipes.	Notebook 06

---

7. Description of Sentence Transformer Embeddings

7.1 Brief Introduction

Sentence Transformer embeddings are dense, low-dimensional vector representations of text sequences generated by fine-tuned pre-trained Transformer language models.

7.2 Detailed Explanation

Unlike bag-of-words or TF-IDF representations, Sentence Transformers map sequences of tokens to a joint semantic vector space where semantically similar texts are mapped to nearby coordinates. The model `all-MiniLM-L6-v2` maps recipe textual documents to 384-dimensional floating-point vectors.

7.3 Examples

Encoding 'Spicy Cajun Chicken Pasta' produces a vector near 'Creamy Cajun Chicken Pasta' (cosine similarity ≈ 0.88), while 'Chocolate Fudge Brownies' maps farther away (cosine similarity ≈ 0.12).

7.4 Advantages

- Captures semantic similarity (e.g., matching "pork" and "bacon") without manual dictionary mapping.
- Fixed 384-dimensional output simplifies downstream database indexing and nearest-neighbor searches.
- Fast scoring via batch matrix dot products.

7.5 Disadvantages

High inference latency: embedding large corpora requires GPU/TPU hardware overhead.

Struggles with domain-specific terms or brand names not seen during model pre-training.

7.6 Use Cases

Semantic search and item retrieval in RAG knowledge bases.

Dense user profile vector representation for content-based recommenders.

Metadata alignment and deduplication in recipe catalog systems.

7.7 Limitations

Cannot capture numerical interaction behaviors (e.g., whether a user liked a recipe, only that it is textually similar).

Truncates documents exceeding token budgets (typically 256–512 tokens).

---

8. Description of Collaborative Filtering

8.1 Brief Introduction

Collaborative Filtering predicts a user's interest in items by leveraging historical interaction patterns across the entire user community.

8.2 Detailed Explanation

Memory-based methods (Cosine KNN) compute explicit user-to-user similarity scores directly on the interaction matrix. Model-based methods (SVD, ALS) factorize the matrix into low-rank latent user and item factor matrices, optimizing parameters to reconstruct observed ratings.

8.3 Examples

If User A and User B both liked "Bourbon Chicken" and "Pancakes", the Cosine KNN model detects high similarity. If User A subsequently rates "Turkey Chowder" highly, the system recommends "Turkey Chowder" to User B.

8.4 Advantages

Domain independence: requires no text metadata or manual feature engineering.

High serendipity: identifies unexpected cross-category overlaps in user preferences.

Self-improving: recommendation accuracy increases as more user transaction logs are added.

8.5 Disadvantages

Cold start: cannot recommend new items or serve new users without historical ratings.

Performance degrades under extreme sparsity (where the interaction matrix is >99.9% empty).

Popularity bias: tends to over-recommend popular items, leaving niche items unrecommended.

8.6 Use Cases

Large-scale e-commerce product recommendations.

Streaming service movie and music suggestion engines.

Bipartite user-item rating prediction in social and content platforms.

8.7 Limitations

Cannot provide content-grounded explanations for its recommendations.

Vulnerable to shilling attacks where fake ratings are injected to promote or demote specific items.

---

9. Comparative Analysis: Content-Based vs. Collaborative Filtering

Content-Based Recommendation	Collaborative Filtering
Leverages item text metadata and descriptive attributes for similarity.	Relies purely on historical transaction logs and user-item matrices.
Avoids item cold-start: new items encoded immediately into the vector space.	Suffers from item cold-start, requiring interactions before recommendations.
Requires a small user interaction history to compute preference vectors.	Can recommend to empty-profile users via global popularity fallback.
Creates semantic filter bubbles (recommends items similar to past history).	Encourages serendipity by identifying cross-category user overlaps.
Projects items to 384-dimensional dense vectors via SentenceTransformers.	Projects interaction matrix into low-rank latent spaces (20 or 64 dims).
Requires intensive feature engineering, tokenization, and text cleaning.	Bypasses feature engineering, operating on rating or implicit labels.

Vector index schemas (HNSW/IVF-Flat) used for similarity search in DB.	Relational indexes on user_id and recipe_id for query resolution.
High retrieval latency due to high-dimensional nearest-neighbor search.	Low latency since recommendations can be precomputed and cached.
Scaling centers on Transformer neural network forward passes (GPU).	Scaling centers on linear algebra libraries (OpenBLAS, MKL) (CPU).
User profile vectors expose dietary preferences, requiring encryption.	Abstract matrix rows protect specific dietary preferences from leaks.
Static verification: cryptographic hash of recipe profiles at ingestion.	Dynamic verification: hash of global model weights at each update.
Non-parametric: only vector encoding and indexing required.	Parametric: optimizes latent factors via SGD (SVD) or ALS.
Recall bounded by semantic overlap of user profiles and item embeddings.	Recall scales with data density and latent factor decomposition quality.
Explains recommendations via semantic similarity to liked items.	Cannot provide content-grounded explanations for recommendations.